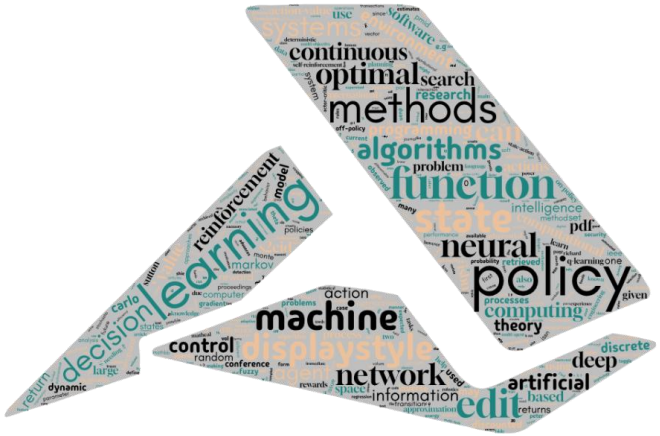


Reinforcement Learning Algorithms and Applications

Chapter 7. Deep Q Learning for Gridworld

Overview and Environment Setup

- **Deep Q-Network Agent** implementation for navigating a 4x4 gridworld environment.
- Agent must learn to reach the **goal** while avoiding **pits** and **walls**.
- Uses **epsilon-greedy strategy** for balancing exploration and exploitation.
- Environment provides **rewards**: +10 for goal, -10 for pit, -1 for each step.
- **State space**: 16 positions (4x4 grid),
- **Action space**: 4 directions (up, down, left, right).
- **Gridworld wrapper** provides standard RL interface for the environment.
- Training involves **1000 episodes** with maximum 200 steps per episode.
- P: Starting position of the agent
- +: Goal state (the agent tries to reach this)
- −: Trap state (the agent should avoid this)
- W: Wall or obstacle that blocks movement

```
[['+' '-' ' ' 'P']
 [' ' 'W' ' ' ' ']
 [' ' ' ' ' ' ' ']
 [' ' ' ' ' ' ' ']]
```

Bellman Equation and Loss Function

- **Q-learning**
  - **Update Rule (Bellman equation):** $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$
  - **Target Q-value:** $r + \gamma \max_{a'} Q(s',a')$
  - **Temporal Difference Error (TD error):** $\underbrace{r + \gamma \max_{a'} Q(s',a')}_{\text{TD target}} - \underbrace{Q(s,a)}_{\text{Current estimate}}$
- **Deep Q-Network**
  - The goal is to minimize the TD error via the following loss function:
  - $L(\theta) = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q(s',a';\theta_{target}) - Q(s,a;\theta)\right)^2\right]$
  - This is essentially a **mean squared error** between:
    - **TD target:**
      - $y = r + \gamma \max_{a'} Q(s',a';\theta_{target})$
      - Computed using the frozen **target network** θ_{target}
    - **Current estimate:**
      - $Q(s,a,\theta)$
      - Output of the **policy network (main network)** θ
  - $\mathbb{E}[\cdot]$: Expectation
    - It means taking the **average over randomly sampled transitions** from the **Replay Buffer**. In practice, this is approximated by a **mini-batch**:
    - In practice, this is approximated by a **mini-batch**:
    - $L(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\left(r_i + \gamma \max_{a'} Q(s_i',a_i';\theta_{target}) - Q(s_i,a_i;\theta)\right)^2$
    - Each iteration, you sample N transitions from the replay buffer, compute the squared TD error for each, average them, and perform one step of gradient descent.

Bellman Equation and Loss Function

- **Deep Q-Network**
  - **Gradient Update**
    - $\nabla_\theta L(\theta) = \mathbb{E}\left[\underbrace{\left(y - Q(s,a;\theta)\right)}_{scalar} \cdot \underbrace{\nabla_\theta Q(s,a;\theta)}_{vector}\right]$
    - The TD target is: $y = r + \gamma \max_{a'} Q\left(s', a'; \theta_{target}\right)$
      - If the TD error is **large** → the gradient step is **big** (big correction needed)
      - If the TD error is **small** → the gradient step is **small** (already close to target)
      - If the TD error is **negative** → the gradient flips direction (we overestimated)
    - **Key point:**
      - y is treated as a **constant** during backpropagation — no gradients flow back through the target network θ_{target}.
      - Only the policy network (main network) θ gets updated.
      - This is why the two networks must be kept separate. If you used the same network for both $y$and $Q(s,a;\theta)$, the target would shift every update, making training unstable.
      - The target network parameters θ_{target} are treated as constants during backpropagation, so gradients only flow through the main network.

Bellman Equation and Loss Function

- **Deep Q-Network**
  - **Deep Q-learning with Experience Replay**

Here, $\phi(s_t) = \text{preprocessing}(s_t)$
$\phi(s_t)$ is the feature representation of s_t.

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights (Both θ_{target} and θ)
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

epsilon-greedy based action selection

Execute action and get reward

V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing Atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, 2013.

DQN Agent Implementation

- **Dual network architecture** maintains main Q-network and target network for stability.
- **Epsilon-greedy exploration** balances exploration and exploitation during training process.
- **State encoding** converts discrete states to one-hot vectors for network input.
- **Action selection formula**: $a = \arg\max_a Q(s, a; \theta)$ for exploitation, random for exploration.
- **Memory storage** saves experiences in replay buffer for batch learning.
- **Target network updates** occur every 100 steps to stabilize training.
- **Hyperparameters** include learning rate (0.001), discount factor (0.95), and batch size (32).

Deep Q-Network Architecture

- **Neural network structure**: Three fully connected layers with ReLU activation functions.
- **Input layer**: State size (16 for 4x4 grid) with one-hot encoding representation.
- **Hidden layers**: Two layers with 64 neurons each for feature extraction.
- **Output layer**: Action size (4) producing Q-values for each possible action.
- **Forward pass equation**: $x = \text{ReLU}(\text{Linear}(x))$ applied sequentially through layers.
- **Network initialization** uses PyTorch's default weight initialization for stable training.
- **Device compatibility** supports both CPU and GPU computation for scalability.

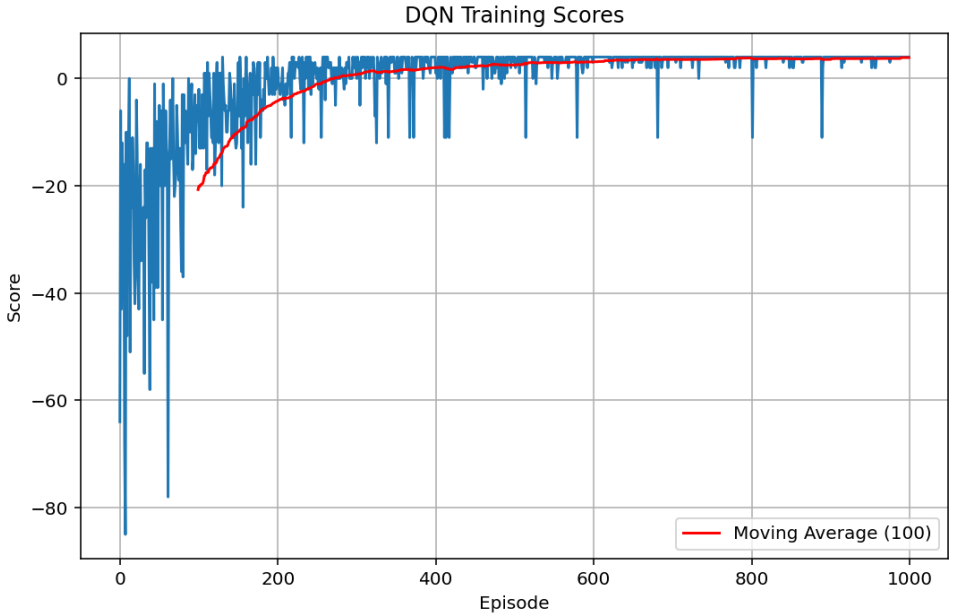
Experience Replay Buffer

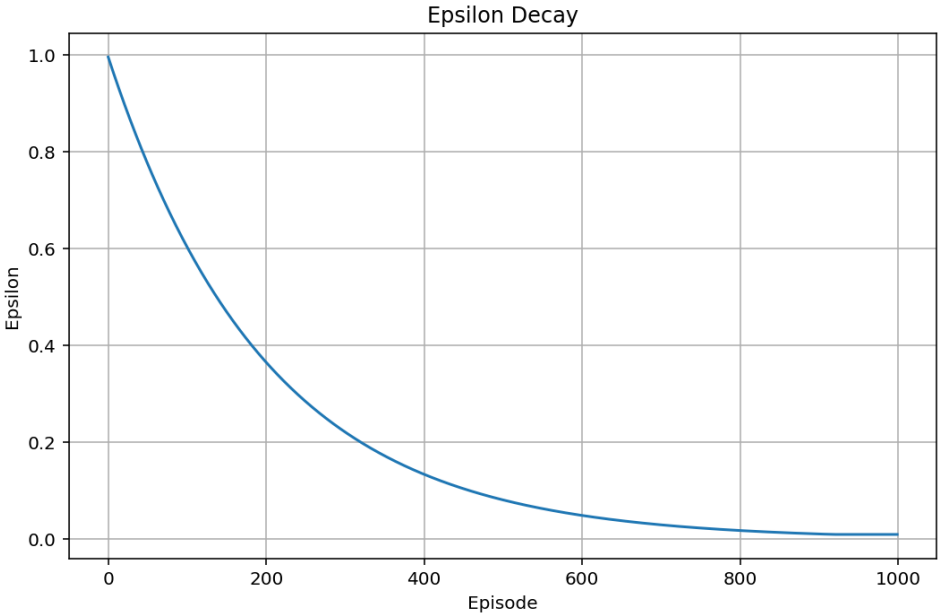
- **Buffer implementation** uses deque with fixed capacity for efficient memory management.
- **Experience storage** contains tuples of (state, action, reward, next_state, done).
- **Sampling method** randomly selects batch_size experiences for training diversity.
- **Data transformation** converts batch data using zip(*batch) for tensor processing.
- **Memory efficiency** automatically removes oldest experiences when capacity exceeded.
- **Batch preparation** converts sampled data to numpy arrays for neural network input.
- **Buffer size** typically set to 10,000 experiences for adequate learning diversity.

Training Process and Results

- **Training loop** runs for 1000 episodes with maximum 200 steps per episode.
- **Experience collection** stores state transitions during environment interaction.
- **Replay learning** samples batches from buffer for neural network training.
- **Epsilon decay** reduces exploration rate from 1.0 to 0.01 over training.
- **Performance tracking** monitors scores, epsilon values, and training progress.
- **Convergence analysis** shows learning curve through moving average calculations.
- **Final evaluation** demonstrates learned policy through test episodes without exploration.

Training Process and Results





Reference

- M. Lapan, *Deep Reinforcement Learning Hands-On*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.
- A. Zai and B. Brown, *Deep Reinforcement Learning in Action*. Shelter Island, NY: Manning Publications, 2020.
- S. Ravichandiran, *Deep Reinforcement Learning with Python*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.
- V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, “Playing Atari with deep reinforcement learning,” *arXiv preprint arXiv:1312.5602*, 2013.